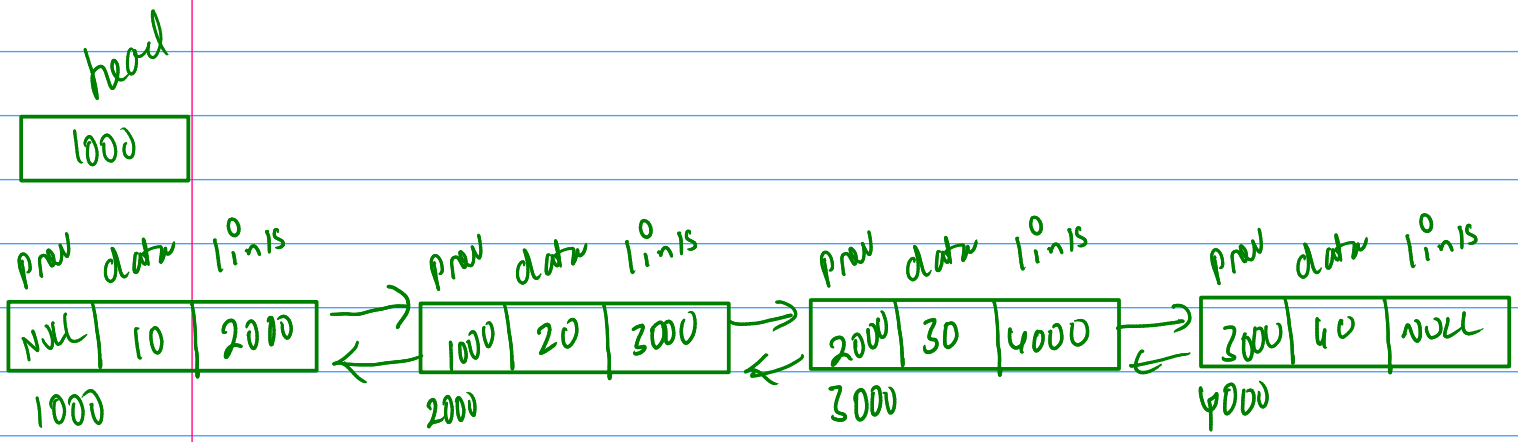
01/04 Doubly linked list

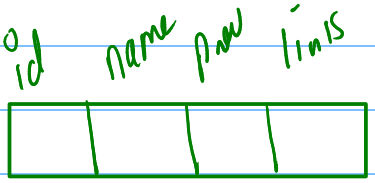
Doubly link list is a linear datastructure which is a collection of nodes where every node is a combination of current node information next node address and previous node address. In Doubly link list we can traverse in forward as well as backward direction.



First node address should be stored in a pointer called head
First node previous should be NULL
Last node link should be NULL

insert / Add
print
delete

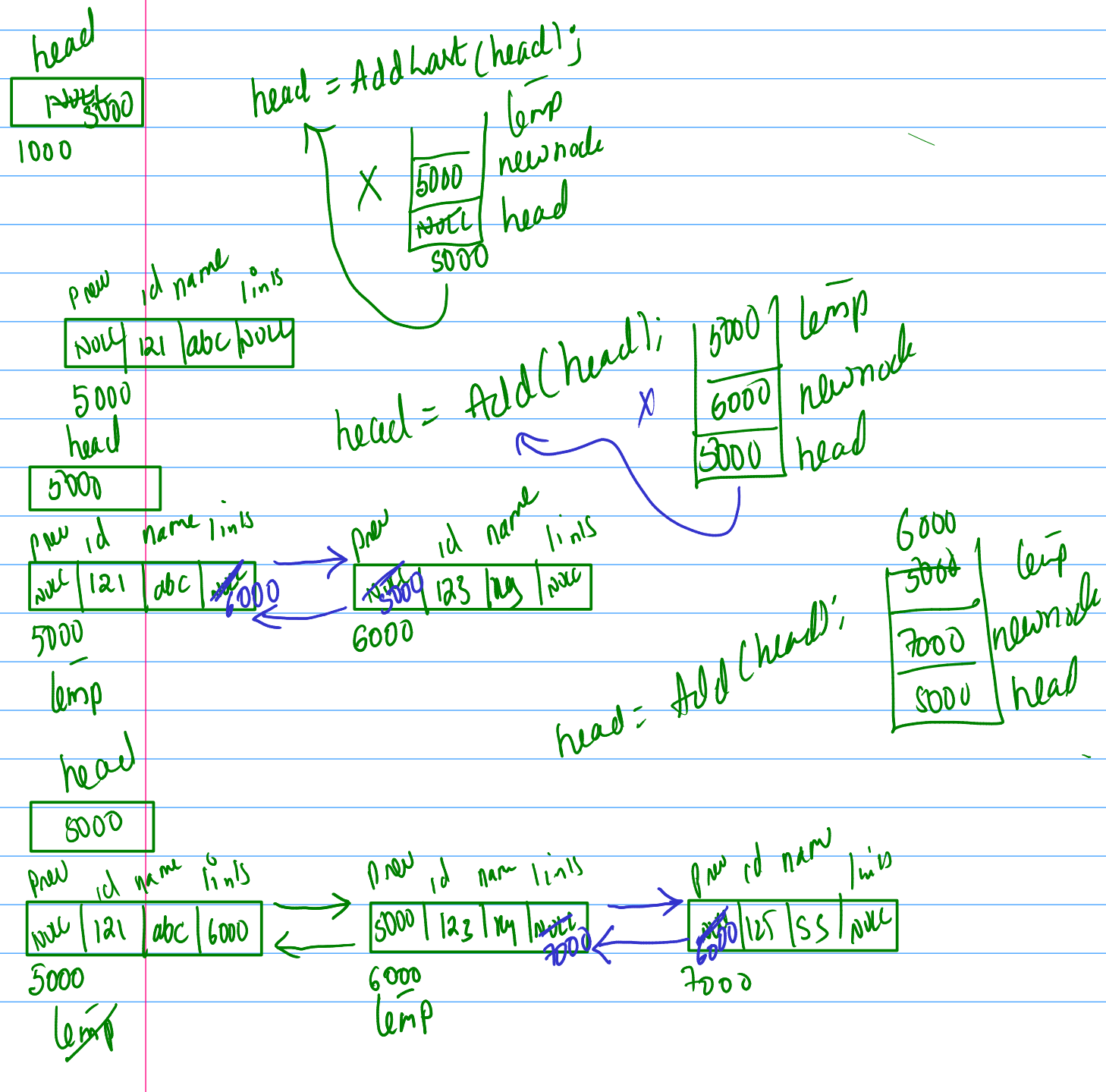
```
struct Employee
{
        int id;
        char name[20];
        struct Employee *prev,*link;
};
```



struct Emp
{ struct Emp *prev;
int id;
char name[20];
struct Emp *link;
};

AddLast

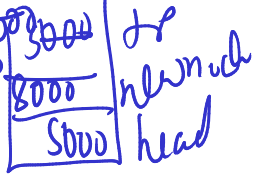
1. create the node
2. initialise the node
3. link to list
    a. if list is empty assign newnode to head
    b. else traverse upto lastnode
    assign newnode address to last node link
    assign lastnode address to newondoe previous
4. return firstnode address

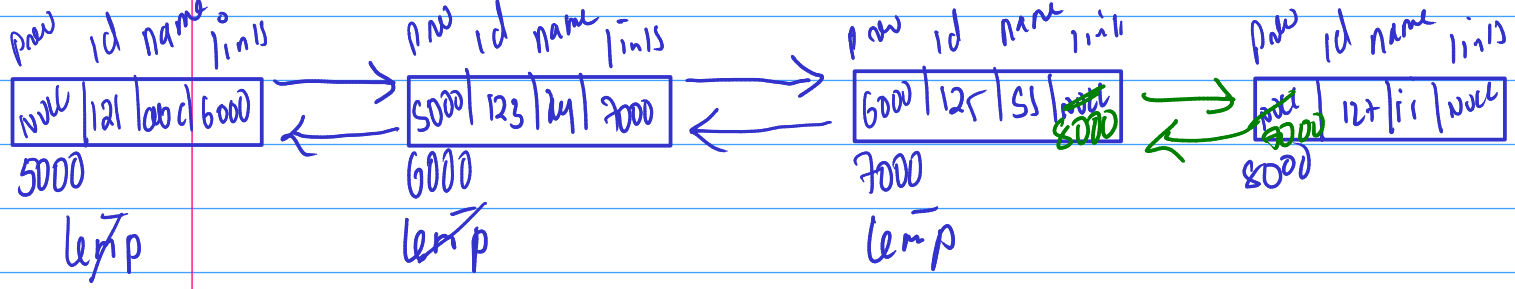


head

5000

head = Add(head);






```
if (head == NULL)
 {  head = newnode;

else
{  for(temp=head; temp->link ; temp = temp->link );

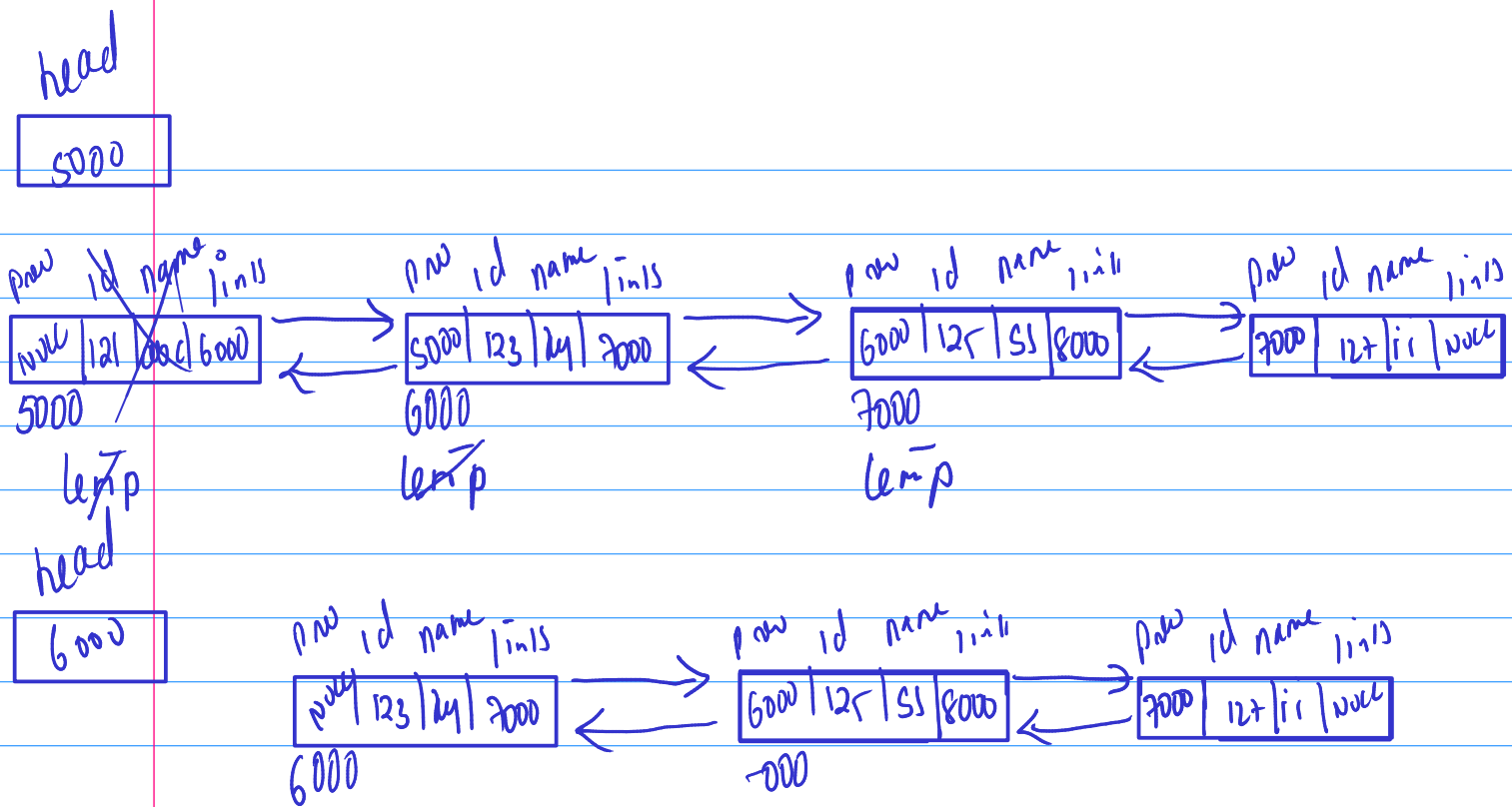
     temp->link = newnode;
     newnode->prev = tp;

return head;
```

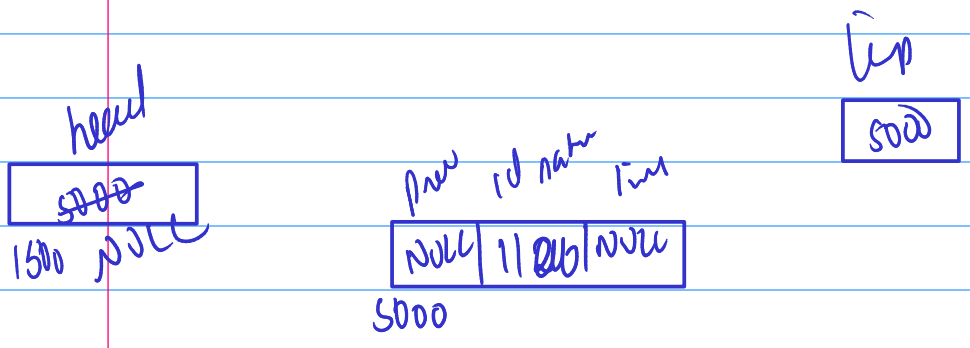
```
void  Rprint ( Employee * head)   or  (head->prev)

{     if (head == NULL)
          List is empty

      else { for( ; head->link; head = head->link);
      while(head)
      { Pf("%d %s\n", head->id, head->name);
                head = head->prev;
      }
```



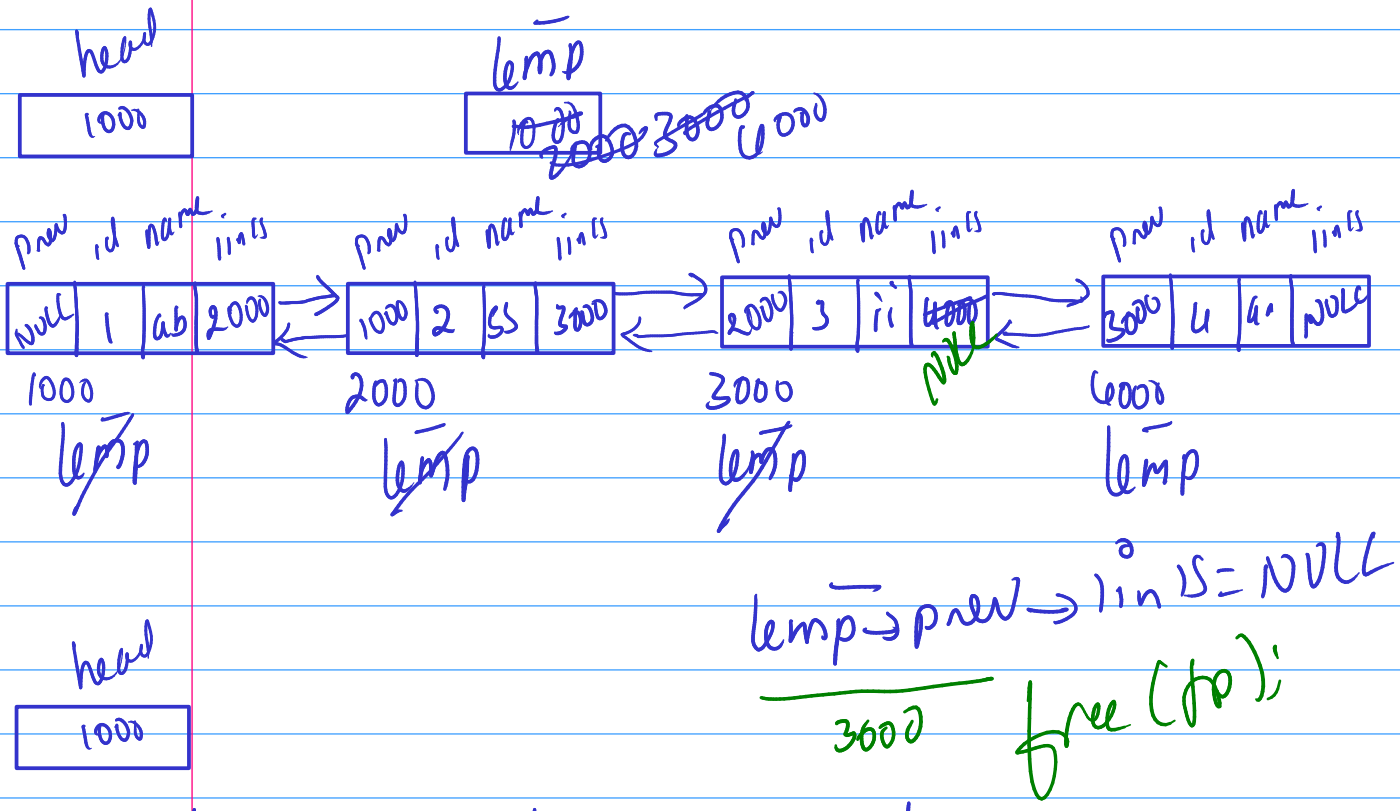
if list is empty , print list empty msg
else
assign head to temp, move head to next node
if head is valid then make head prev NULL
then freee temp
return head

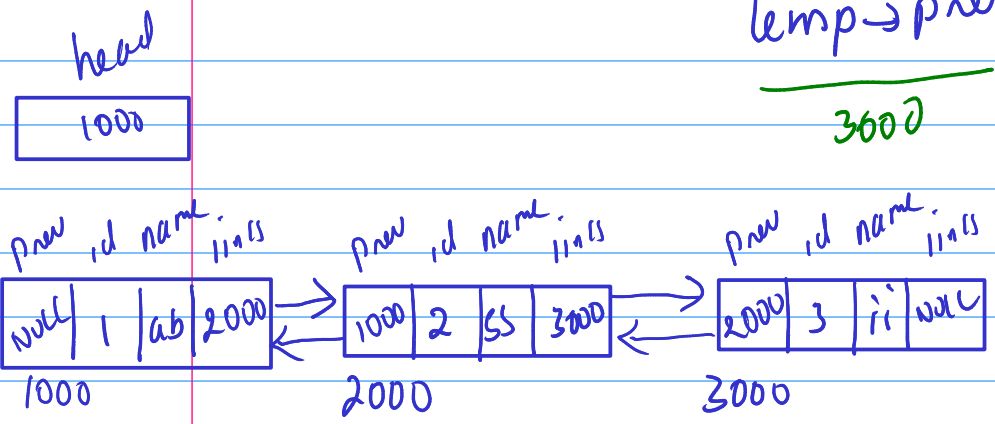



```
if(head==NULL)
        empty
else
{   temp=head;
        head=head->link;
        if(head)
                head->p[rev=NULL
free(temp);     return head;
```

Delete Last Node

list is empty , print empty msg
else if list having only one node delete head and assign NULL to head
else
traverse upto last node
assign NULL to last before node
then free lastnode

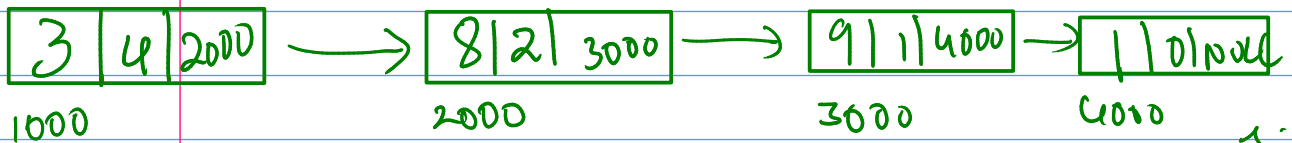




$3x^4 + 9x + 8x^1 + 1$

struct poly
{ int coeff;
int exp;
struct poly *link;
};

$3x^4 + 8x^2 + 9x + 1$



$7x^3 + 3x^2 + 5x + 9$

